Database

為什麼資料庫的選擇這麼複雜

打開任何一份系統設計的技術文件，你都會看到一串資料庫名字：PostgreSQL、DynamoDB、Redis、Cassandra、MongoDB、ClickHouse、Pinecone……它們都是「資料庫」，但設計哲學完全不同。

這不是因為軟體工程師喜歡造輪子。而是因為「儲存和查詢資料」這件事，在不同的業務場景下，本質上是完全不同的問題——點查詢和全表掃描的最佳儲存結構截然相反；強一致性和高擴展性在工程上存在根本的張力；用向量距離做相似度搜尋和用主鍵做精確查詢，需要的索引結構也完全不一樣。

這篇講義是整個資料庫系列的入口。目標不是讓你記住每個資料庫的 feature list，而是建立一張**地圖**——理解不同資料庫存在的原因，以及面試時如何快速定位到正確的選擇。

兩個維度，看清資料庫的全景

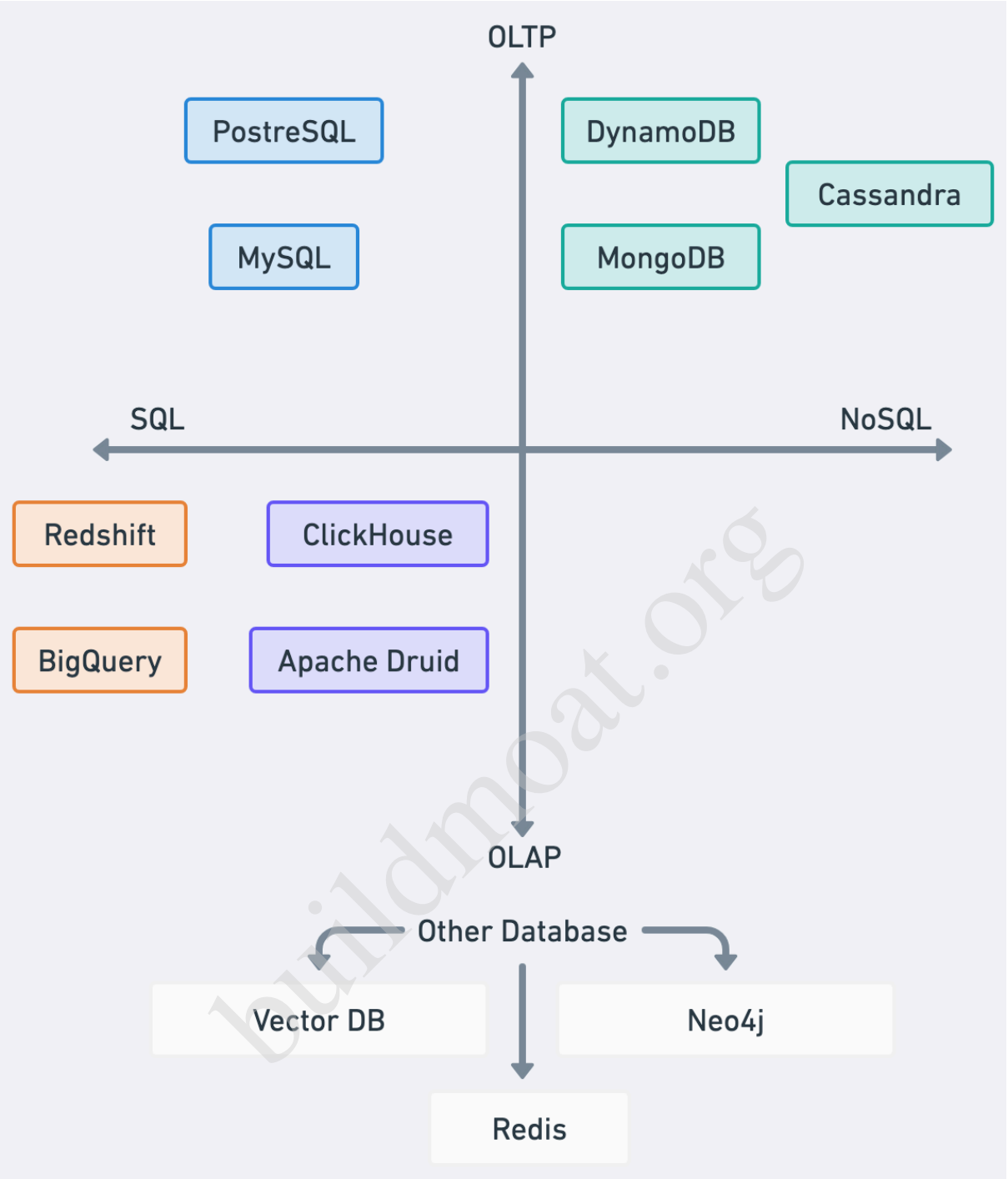
理解資料庫，有兩個維度最重要，而且它們是**正交的**（互相獨立）：

維度一：資料模型（Data Model）
資料長什麼形狀？是有固定 schema 的表格，還是靈活的文件，還是圖結構？

維度二：工作負載（Workload）
這個資料庫要處理什麼樣的查詢？是高並發的點查詢和小量寫入（OLTP），還是對海量歷史資料做聚合分析（OLAP）？

把這兩個維度疊在一起，大多數資料庫的定位就清楚了：

buildnoat.org



面試時遇到資料庫選型的問題，腦袋裡先跑這兩個問題：「資料是什麼形狀？」「這是交易型還是分析型的查詢？」答案通常就指向了正確的方向。

關聯式資料庫（RDBMS）

關聯式資料庫是最成熟、最廣泛使用的資料庫類型。資料以表格（Table）形式存在，每一行（Row）是一筆記錄，每一列（Column）是一個屬性。表格之間用 **Foreign**

Key 建立關聯，用 **SQL** 查詢。

ACID 是核心承諾

RDBMS 最重要的特性是 **ACID 事務**：

- **Atomicity（原子性）**：「扣庫存 + 建訂單 + 記付款」是一個不可分割的整體，要麼全成功，要麼全回滾
- **Consistency（一致性）**：事務前後，所有的 constraint（外鍵、唯一鍵）都必須滿足
- **Isolation（隔離性）**：並發的事務互不干擾，不會看到彼此的中間狀態
- **Durability（持久性）**：commit 之後，即使伺服器崩潰，資料也不會遺失

這四個性質讓 RDBMS 成為業務交易的天然選擇：任何涉及錢、庫存、狀態轉換的操作，出錯了要能乾淨地回滾，這正是 ACID 解決的問題。

強大的查詢能力

SQL 是表達力強的查詢語言，JOIN、聚合、子查詢、視窗函數，複雜的資料關係用 SQL 幾行就能表達，換成 NoSQL 可能需要在應用層寫大量程式碼。

```
-- 找出上個月訂單金額最高的前 10 個用戶，以及他們的總消費
SELECT u.name, SUM(o.amount) AS total_spent
FROM users u
JOIN orders o ON u.id = o.user_id
WHERE o.created_at >= DATE_TRUNC('month', NOW() - INTERVAL
'1 month')
GROUP BY u.id, u.name
ORDER BY total_spent DESC
LIMIT 10;
```

RDBMS 的限制

Schema 是雙面刃：事先定義結構讓資料整齊可靠，但需求變動時改 schema（加欄、改型別）在大資料量下成本很高。

水平擴展困難：RDBMS 天生適合垂直擴展（買更大的機器）。水平擴展（分散到多台機器）需要 Sharding，而跨節點的事務和 JOIN 會大幅增加複雜度。大多數 RDBMS 沒有原生的 Sharding 方案。

深入講義：PostgreSQL 涵蓋索引設計、讀寫效能調校、複製與高可用。

buildboat.org

NoSQL 資料庫

NoSQL（Not Only SQL）不是一個單一的東西，而是一系列**用不同資料模型換取不同取捨**的資料庫的統稱。它們共同的特點是：放棄（或弱化）關聯式模型和 SQL，換取更高的擴展性或更靈活的資料結構。

Key-Value Store

最簡單的模型：一個 key 對應一個 value，沒有 schema，沒有查詢語言，只有 get/set/delete。

極低的延遲（Redis 通常在微秒級），極高的吞吐量。適合快取、Session 管理、計數器、排行榜。

Redis 是最廣泛使用的 Key-Value Store，同時支援豐富的資料結構（List、Set、Sorted Set、Hash），讓它能做的事遠超過單純的快取。

DynamoDB 是 AWS 的全託管 Key-Value（兼 Document）Store，設計給極高併發、低延遲的大規模 OLTP 場景。

深入講義：DynamoDB 涵蓋 partition key 設計、GSI/LSI、定價模型。

Document Store

以 JSON（或類 JSON）文件為基本單位。每筆記錄是一個文件，可以有巢狀結構，同一個集合（Collection）裡的文件不需要有相同的欄位。

適合資料結構多變、快速迭代的場景：用戶設定檔、內容管理系統、產品目錄。

MongoDB 是最主流的 Document Store，支援豐富的查詢語法，讓你能夠查詢嵌套欄位、做陣列查詢，比純 Key-Value 靈活很多。

Wide-Column Store

以「列族（Column Family）」為單位組織資料。每一行（Row）可以有不同的欄位組合，欄位在磁碟上按列族連續排放。

適合高寫入吞吐量、時序資料、有明確 access pattern 的大規模 OLTP 場景。

Cassandra 的設計哲學是：write anywhere，用最終一致性換取超高的寫入吞吐量和無單點故障。它是 OLTP 系統，不是分析系統——它按行儲存，適合高並發的寫入和有明確 primary key 的查詢，不適合做跨行的聚合分析。

Graph Database

把資料表示為節點（Node）**和**邊（Edge）的圖結構，讓「關係」本身成為一等公民。適合社交網路（誰認識誰）、推薦系統（用戶 A 喜歡的商品，和喜歡商品 B 的用戶有

buildroot.org

幾層關係）、知識圖譜。

在高度連結的資料上，圖資料庫的多跳（multi-hop）查詢比 SQL 的多層 JOIN 快幾個數量級。

Neo4j 是最主流的圖資料庫，使用 Cypher 查詢語言。

NoSQL 的共同取捨

NoSQL 資料庫通常遵循 **BASE** 模型，而非 ACID：

- **Basically Available**：系統大多數時候是可用的，允許部分節點故障
- **Soft state**：系統的狀態可能隨時間改變（即使沒有新的寫入）
- **Eventual consistency**：資料最終會收斂到一致，但不保證任何時刻都一致

這讓 NoSQL 能做到 RDBMS 很難做到的水平擴展——當一致性的要求放鬆了，資料就可以分散到多個節點，每個節點獨立處理請求，不需要跨節點的協調。

OLTP vs OLAP

這是另一個獨立的分類維度，和 RDBMS vs NoSQL 正交。

OLTP（Online Transactional Processing）：為業務交易設計。高並發的點查詢和小量寫入，毫秒級延遲，需要 ACID 保證。PostgreSQL、DynamoDB、Cassandra 都是 OLTP 系統。

OLAP（Online Analytical Processing）：為分析查詢設計。對海量歷史資料做聚合運算，單次查詢可能掃描幾億行，幾秒到幾分鐘的延遲可以接受。使用行式儲存（Column-oriented storage）讓只讀幾個欄位的聚合查詢大幅減少 I/O。BigQuery、Redshift、ClickHouse、Snowflake 是 OLAP 系統。

在實際架構裡，業務資料先進入 OLTP，再透過 ETL 或 CDC 管線搬到 OLAP，讓分析查詢完全不影響線上業務的效能。

深入講義：OLTP vs OLAP 涵蓋行式儲存的底層原理、星型 Schema、ETL vs CDC 的取捨、HTAP。

Vector Database

向量資料庫是近年因為機器學習和 LLM 興起而快速成長的特殊類別。

傳統資料庫做的是精確查詢：「找出 user_id = 123 的訂單」。向量資料庫做的是**相似度搜尋**：「找出和這段文字語意最接近的 10 筆文件」。

這需要把非結構化資料（文字、圖片、音訊）轉換成高維的**向量（Embedding）**，然後在向量空間裡搜尋距離最近的鄰居（Approximate Nearest Neighbor, ANN）。

buildmoat.org

```
用戶輸入：「我想找一雙適合長距離跑步的鞋」
        ↓ Embedding 模型
向量：[0.23, -0.71, 0.45, 0.89, ...]（1536 維）
        ↓ ANN 搜尋
找出向量空間中距離最近的商品文件
        ↓
結果：慢跑鞋 A、路跑鞋 B、馬拉松鞋 C
```

常見的應用場景：語意搜尋、RAG（Retrieval-Augmented Generation）、以圖搜圖、推薦系統。

獨立向量資料庫：Pinecone、Weaviate、Milvus——專門為向量搜尋設計，提供完整的向量管理和搜尋 API。

整合向量能力的現有資料庫：pgvector（PostgreSQL 擴充套件）、Redis VSS、Elasticsearch——讓你在現有的資料庫裡直接做向量搜尋，不需要引入新系統。

深入講義：Vector DB 的詳細設計見 VectorDB。

在面試中如何選擇資料庫

不要一開始就比較 SQL vs NoSQL

這是最常見的誤區。面試官想看到的是你從需求推導出選擇的過程，而不是背誦比較表。

好的思路是依序問自己三個問題：

第一：資料是什麼形狀、有什麼查詢模式？

- 有固定結構、需要複雜 JOIN → RDBMS（PostgreSQL）
- Key-Value 存取、需要超低延遲 → Key-Value Store（DynamoDB、Redis）
- 文件型、schema 需要靈活演進 → Document Store（MongoDB）
- 高度連結的關係資料 → Graph DB（Neo4j）
- 非結構化資料、需要相似度搜尋 → Vector DB

第二：是 OLTP 還是 OLAP？

- 業務交易、高並發、需要 ACID → OLTP
- 歷史資料分析、聚合查詢 → OLAP（考慮獨立的資料倉儲）

第三：規模和一致性的取捨？

buildnoat.org

- 強一致性 + 複雜查詢 → RDBMS
- 需要輕鬆水平擴展、最終一致性可接受 → NoSQL

說出你的理由

選了資料庫之後，一定要說明原因：

- 選 **PostgreSQL** → 強調 ACID 事務、複雜的 JOIN 查詢、成熟的生態
- 選 **DynamoDB** → 強調低延遲、大規模水平擴展、managed service 降低運維成本
- 選 **Cassandra** → 強調極高的寫入吞吐量、多地區部署、寫入不怕單點故障
- 選 **BigQuery / Redshift** → 強調和 OLTP 分離、行式儲存讓分析查詢快幾個數量級

什麼時候不需要複雜的資料庫

- **MVP / 小型系統**：SQLite 或單一 PostgreSQL 就夠了，不要過度工程化
- **單純的快取需求**：Redis 或 Memcached，不需要 MongoDB
- **單純的檔案儲存**：圖片、影片、大型文件用 Blob Storage，不要塞進資料庫

buildmoat.org